Assignment 3: Autonomous Multi-Doc Financial Analyst

本次作業的目標是建立一個能夠分析多份財務文件的 Autonomous Multi-Doc Financial Analyst。系統需根據 Apple 與 Tesla 的財務報表回答使用者問題，並比較 LangChain 與 LangGraph 兩種實作方式的差異。

在本專案中，我以助教提供的 sample code 為基礎，完成文件檢索式問答流程，並進一步實作較具彈性與可控性的 LangGraph 架構，以提升系統面對不同題型時的穩定性。此外，我也比較了不同 embedding model 與 chunk size 的效果，分析其對檢索與回答品質的影響。

系統架構

本專案主要包含兩套問答系統：LangChain Legacy Agent 與 LangGraph Agent。

（一）LangChain Legacy Agent

LangChain 版本採用 ReAct（Reasoning + Acting）風格的 agent。系統透過自訂 prompt，要求模型依照 Thought、Action、Action Input、Observation、Final Answer 的格式執行，並搭配 retriever tool 進行財務文件檢索。此版本的優點是架構相對簡單，適合作為 baseline。

（二）LangGraph Agent

LangGraph 版本則將整體流程拆解為多個節點，包含 Router、Retriever、Grader、Rewriter 與 Generator。當問題進入系統後，會先由 Router 判斷應查詢 Apple、Tesla、兩者皆查，或是與資料集無關。接著再根據檢索結果決定是否可直接生成答案，或需要先改寫問題後重新檢索。

此外，為了處理 compare 題型，我額外將 Apple 與 Tesla 的檢索流程拆成雙支線，使系統能分別檢查兩家公司資料是否足夠，再進一步合併產生比較答案。

資料處理與向量資料庫建置

本專案的資料來源為 Apple 與 Tesla 的財務 PDF 文件。整體資料處理流程主要以助教提供的 sample code 為基礎，再配合本次實驗需求進行調整與測試。

在前處理部分，系統會先使用 PDF loader 讀取文件內容，並進行基本文字清理，例如將多餘換行與空白統一處理，以降低後續切塊與檢索時的雜訊。之後再利用 RecursiveCharacterTextSplitter 將文件切成多個 chunk，並透過 embedding model 將各 chunk 轉換為向量表示，最後存入 Chroma 向量資料庫。

此外，本專案分別為 Apple 與 Tesla 建立獨立的 vector database，讓後續系統可根據問題所屬公司選擇對應 retriever 進行檢索。這樣的設計有助於降低不同公司文件彼此混淆的情況，也方便後續 compare 題型的擴充與處理。

LangGraph 實作流程

（一）Intelligent Router

Router 的功能是將使用者問題分類為四種類型：apple、tesla、both、none。若問題只詢問 Apple，則只查 Apple 的向量資料庫；若只詢問 Tesla，則只查 Tesla；若問題是比較題，則進入雙支線流程；若問題與資料集無關，則不進行無意義檢索。

（二）Relevance Grader

檢索完成後，系統會利用 grader 判斷目前取得的文件內容是否足以回答問題。grader 只輸出 yes 或 no，以維持判斷流程簡單明確。若內容充足，系統即可進入答案生成；若內容不足，則會進入 query rewriting 階段。

（三）Query Rewriter

當問題本身太模糊，或檢索結果不夠精準時，系統會改寫問題。例如將「new tech spending」改寫成較常出現在財報中的「research and development expenses」，讓 retriever 更容易找到對應資訊。此設計可提升對模糊問題的處理能力。

（四）Final Generator

Generator 的任務是根據檢索到的內容產生最終答案。此模組被明確限制只能使用提供的 context，不可依賴模型的外部知識，且若證據不足時必須誠實回答 I don't know，而不能猜測。此外，答案也必須附上來源標註。

（五）Compare 題雙支線流程

為了處理 compare 題型，我將 LangGraph 流程延伸為 Apple 與 Tesla 雙支線。系統在 compare 題時，會先進入 compare branch，再分別對 Apple 與 Tesla 執行 retrieval、grading 與必要時的 rewriting，最後才合併兩邊 context 產生比較答案。

實驗設計與結果分析

Embedding Model 比較

本專案共比較了兩個 embedding model：

1. sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
2. sentence-transformers/all-MiniLM-L6-v2

第一個 model

Q1

```
>>> print("Q1:", run_graph_agent("What was Apple's revenue in 2024?"))
--- 🔍 RETRIEVING ---
🎯 Routing to: apple
   Query for apple: What was Apple's revenue in 2024?
Failed to send telemetry event CollectionQueryEvent: capture() takes 1 positional argument but 3 were given
--- ⚖️ GRADING ---
   Relevance Grade: yes
--- ✍️  GENERATING ---
Q1: Apple's revenue in 2024 was $391,035 million [Source: Apple].
```

Q2

```
>>> print("Q2:", run_graph_agent("How much did Apple spend on new tech in 2024?"))
--- 🔍 RETRIEVING ---
🎯 Routing to: apple
   Query for apple: How much did Apple spend on new tech in 2024?
--- ⚖️ GRADING ---
   Relevance Grade: no
--- 🔄 REWRITING QUERY ---
   New Question: What were Apple's research and development expenses in 2024?
--- 🔍 RETRIEVING ---
🎯 Routing to: apple
   Query for apple: What were Apple's research and development expenses in 2024?
--- ⚖️ GRADING ---
   Relevance Grade: yes
--- ✍️ GENERATING ---
Q2: Apple's research and development expenses in 2024 were $31,370 million [Source: Apple].
```

Q3

```
>>> print("Q3:", run_graph_agent("Compare Apple and Tesla revenue in 2024."))
--- 🔍 RETRIEVING ---
🎯 Routing to: both
--- 🛠️ PREPARING COMPARE QUERIES ---
   Apple query: Apple revenue in 2024.
   Tesla query: Tesla revenue in 2024.
--- 🍎 RETRIEVING APPLE FOR COMPARISON ---
   Apple query: Apple revenue in 2024.
--- 🍎⚖️ GRADING APPLE FOR COMPARISON ---
   Apple Grade: yes
--- 🚗 RETRIEVING TESLA FOR COMPARISON ---
   Tesla query: Tesla revenue in 2024.
--- 🚗⚖️ GRADING TESLA FOR COMPARISON ---
   Tesla Grade: yes
--- ⚖️✍️ GENERATING COMPARISON ANSWER ---
Q3: Apple's total net sales in 2024 were $391,035 million, while Tesla's total revenues in 2024 were $97,690 million. Ap
ple had significantly higher revenue than Tesla in 2024 [Source: Apple] [Source: Tesla].
```

Q4

```
>>> print("Q4:", run_graph_agent("What was NVIDIA's revenue in 2024?"))
--- 🔍 RETRIEVING ---
🎯 Routing to: none
--- ⚖️ GRADING ---
   Relevance Grade: no (empty context)
--- 🔄 REWRITING QUERY ---
   New Question: What was NVIDIA's total revenue in 2024?
--- 🔍 RETRIEVING ---
🎯 Routing to: none
--- ⚖️ GRADING ---
   Relevance Grade: no (empty context)
--- 🔄 REWRITING QUERY ---
   New Question: What was NVIDIA's total revenue for the fiscal year 2024?
--- 🔍 RETRIEVING ---
🎯 Routing to: none
--- ⚖️ GRADING ---
   Relevance Grade: no (empty context)
   (Max retries reached, generating anyway...)
--- ✍️ GENERATING ---
Q4: I don't know.
```

第二個 model

Q1

```
>>> print("Q1:", run_graph_agent("What was Apple's revenue in 2024?"))
--- 🔍 RETRIEVING ---
🎯 Routing to: apple
   Query for apple: What was Apple's revenue in 2024?
Failed to send telemetry event CollectionQueryEvent: capture() takes 1 positional argument but 3 were given
--- ⚖️ GRADING ---
   Relevance Grade: yes
--- ✍️ GENERATING ---
Q1: Apple's revenue in 2024 was $391,035 million [Source: Apple].
```

Q2

```
>>> print("Q2:", run_graph_agent("How much did Apple spend on new tech in 2024?"))
--- 🔍 RETRIEVING ---
🎯 Routing to: apple
   Query for apple: How much did Apple spend on new tech in 2024?
--- ⚖️ GRADING ---
   Relevance Grade: no
--- 🔄 REWRITING QUERY ---
   New Question: What were Apple's research and development expenses in 2024?
--- 🔍 RETRIEVING ---
🎯 Routing to: apple
   Query for apple: What were Apple's research and development expenses in 2024?
--- ⚖️ GRADING ---
   Relevance Grade: yes
--- ✍️ GENERATING ---
Q2: Apple's research and development expenses in 2024 were $31,370 million [Source: Apple].
```

Q3

```
>>> print("Q3:", run_graph_agent("Compare Apple and Tesla revenue in 2024."))
--- 🔍 RETRIEVING ---
🎯 Routing to: both
--- 🧩 PREPARING COMPARE QUERIES ---
   Apple query: Apple revenue in 2024.
   Tesla query: Tesla revenue in 2024.
--- 🍎 RETRIEVING APPLE FOR COMPARISON ---
   Apple query: Apple revenue in 2024.
--- 🍎⚖️ GRADING APPLE FOR COMPARISON ---
   Apple Grade: yes
--- 🚗 RETRIEVING TESLA FOR COMPARISON ---
   Tesla query: Tesla revenue in 2024.
--- 🚗⚖️ GRADING TESLA FOR COMPARISON ---
   Tesla Grade: yes
--- ⚖️✍️ GENERATING COMPARISON ANSWER ---
Q3: Apple's total net sales for the twelve months ended September 28, 2024, were $391,035 million, while Tesla's total r
evenues for the year ended December 31, 2024, were $97,690 million. Apple had significantly higher revenue than Tesla in
 2024 [Source: Apple] [Source: Tesla].
```

Q4

```
>>> print("Q4:", run_graph_agent("What was NVIDIA's revenue in 2024?"))
--- 🔍 RETRIEVING ---
🎯 Routing to: none
--- ⚖️ GRADING ---
   Relevance Grade: no (empty context)
--- 🔄 REWRITING QUERY ---
   New Question: What was NVIDIA's total revenue in 2024?
--- 🔍 RETRIEVING ---
🎯 Routing to: none
--- ⚖️ GRADING ---
   Relevance Grade: no (empty context)
--- 🔄 REWRITING QUERY ---
   New Question: What was NVIDIA's total revenue for the fiscal year 2024?
--- 🔍 RETRIEVING ---
🎯 Routing to: none
--- ⚖️ GRADING ---
   Relevance Grade: no (empty context)
   (Max retries reached, generating anyway...)
--- ✍️ GENERATING ---
Q4: I don't know.
```

實驗結果顯示，這兩個模型在本次作業中的差異不大。無論是一般單一公司問題，或是 Apple 與 Tesla 的比較問題，兩者大多都能檢索到足夠的相關內容，最終回答品質也相近。

我認為造成差異不明顯的原因，可能是本次資料集規模相對較小，且文件主題高度集中在 Apple 與 Tesla 的財務報表，因此檢索難度不像開放領域問題那麼高。在此情況下，只要 embedding model 具備基本語意表示能力，通常都已足以支撐檢索需求。

另外，雖然 paraphrase-multilingual-MiniLM-L12-v2 具有多語言特性，但本作業主要處理的是英文財務文件，最終答案也要求使用英文，因此多語言能力在本次情境中並未帶來特別明顯的優勢。

綜合四個測試題型的結果來看，兩個 embedding model 在本作業中的行為差異有限，顯示在目前文件數量較少且主題集中時，embedding model 的影響不如整體流程設計來得明顯。

Chunk Size 比較

本次實驗比較的 chunk size 為 1000、2000 與 4000。這部分的重點不只是觀察整體回答結果有沒有差異，更重要的是分析不同 chunk size 對大型財務表格問答能力的影響。

在財務文件中，像是 Balance Sheet、Income Statement、Cash Flow Statement 這類表格通常包含多列與多欄資訊，若切塊方式不合適，便可能造成同一張表中的年份、欄位名稱與數值被拆散到不同 chunk，進而影響檢索與回答品質。因此，chunk size 的調整本質上是在權衡 Context Precision 與 Context Completeness。

1. chunk size = 1000
較小的 chunk 通常具有較高的 Context Precision。因為每個 chunk 範圍較小、內容較集中，retriever 比較容易抓到和問題最相關的片段，也較不容易夾帶過多無關資訊。然而，在大型表格情況下，小 chunk 也更容易把原本應一起出現的欄位名稱、年份與金額拆開。例如模型可能只看到某個數值，卻沒有完整看到該數值所對應的欄位名稱或年份，導致回答時上下文不足。

2. chunk size = 2000
chunk size = 2000 在 precision 與 completeness 之間呈現較平衡的效果。相較於 1000，它較能保留同一段表格上下文，降低關鍵欄位被切散的機率；相較於 4000，又不至於讓單一 chunk 過大而混入太多雜訊。以本次作業的資料規模與題型來看，2000 屬於較折衷且穩定的設定。

3. chunk size = 4000
較大的 chunk 具有較高的 Context Completeness，特別適合保留大型財務表格的完整結構。對於需要同時看到年份、科目名稱與數值的題目，大 chunk 較有機會一次提供足夠上下文，使模型更容易理解整體表格內容。不過，缺點是單一 chunk 中的資訊量較多，可能同時夾帶較多不相關文字，導致檢索精準度下降，也可能增加模型在生成答案時處理雜訊的負擔。

chunk size = 1000

```
>>> print("Q1:", run_graph_agent("What was Apple's revenue in 2024?"))
--- 🔍 RETRIEVING ---
🎯 Routing to: apple
   Query for apple: What was Apple's revenue in 2024?
Failed to send telemetry event CollectionQueryEvent: capture() takes 1 positional argument but 3 were given
--- ⚖️ GRADING ---
   Relevance Grade: yes
--- ✍️ GENERATING ---
Q1: Apple's revenue in 2024 was $391,035 million [Source: Apple].
```

```
>>> print("Q2:", run_graph_agent("How much did Apple spend on new tech in 2024?"))
--- 🔍 RETRIEVING ---
🎯 Routing to: apple
   Query for apple: How much did Apple spend on new tech in 2024?
--- ⚖️ GRADING ---
   Relevance Grade: no
--- 🔄 REWRITING QUERY ---
   New Question: What were Apple's research and development expenses in 2024?
--- 🔍 RETRIEVING ---
🎯 Routing to: apple
   Query for apple: What were Apple's research and development expenses in 2024?
--- ⚖️ GRADING ---
   Relevance Grade: yes
--- ✍️ GENERATING ---
Q2: Apple's research and development expenses in 2024 were $31,370 million [Source: Apple].
>>> print("Q3:", run_graph_agent("Compare Apple and Tesla revenue in 2024."))
--- 🔍 RETRIEVING ---
🎯 Routing to: both
--- 🧩 PREPARING COMPARE QUERIES ---
   Apple query: Apple revenue in 2024.
   Tesla query: Tesla revenue in 2024.
--- 🍎 RETRIEVING APPLE FOR COMPARISON ---
   Apple query: Apple revenue in 2024.
--- 🍎⚖️ GRADING APPLE FOR COMPARISON ---
   Apple Grade: yes
--- 🚗 RETRIEVING TESLA FOR COMPARISON ---
   Tesla query: Tesla revenue in 2024.
Failed to send telemetry event CollectionQueryEvent: capture() takes 1 positional argument but 3 were given
--- 🚗⚖️ GRADING TESLA FOR COMPARISON ---
   Tesla Grade: yes
--- ⚖️✍️ GENERATING COMPARISON ANSWER ---
Q3: Apple's total net sales for the twelve months ended September 28, 2024, were $391,035 million. Tesla's total rev
enues for the year ended December 31, 2024, were $97,690 million. Apple had significantly higher revenue than Tesla
in 2024 [Source: Apple] [Source: Tesla].
```

```
>>> print("Q4:", run_graph_agent("What was NVIDIA's revenue in 2024?"))
--- 🔍 RETRIEVING ---
🎯 Routing to: none
--- ⚖️ GRADING ---
   Relevance Grade: no (empty context)
--- 🔄 REWRITING QUERY ---
   New Question: What was NVIDIA's total revenue in 2024?
--- 🔍 RETRIEVING ---
🎯 Routing to: none
--- ⚖️ GRADING ---
   Relevance Grade: no (empty context)
--- 🔄 REWRITING QUERY ---
   New Question: What was NVIDIA's total revenue for the fiscal year 2024?
--- 🔍 RETRIEVING ---
🎯 Routing to: none
--- ⚖️ GRADING ---
   Relevance Grade: no (empty context)
   (Max retries reached, generating anyway...)
--- ✍️ GENERATING ---
Q4: I don't know.
```

Chunk size = 2000

```
>>> print("Q1:", run_graph_agent("What was Apple's revenue in 2024?"))
--- 🔍 RETRIEVING ---
🎯 Routing to: apple
   Query for apple: What was Apple's revenue in 2024?
Failed to send telemetry event CollectionQueryEvent: capture() takes 1 positional argument but 3 were given
--- ⚖️ GRADING ---
   Relevance Grade: yes
--- ✍️ GENERATING ---
Q1: Apple's revenue in 2024 was $391,035 million [Source: Apple].
>>> print("Q2:", run_graph_agent("How much did Apple spend on new tech in 2024?"))
--- 🔍 RETRIEVING ---
🎯 Routing to: apple
   Query for apple: How much did Apple spend on new tech in 2024?
--- ⚖️ GRADING ---
   Relevance Grade: no
--- 🔄 REWRITING QUERY ---
   New Question: What were Apple's research and development expenses in 2024?
--- 🔍 RETRIEVING ---
🎯 Routing to: apple
   Query for apple: What were Apple's research and development expenses in 2024?
--- ⚖️ GRADING ---
   Relevance Grade: yes
--- ✍️ GENERATING ---
Q2: Apple's research and development expenses in 2024 were $31,370 million [Source: Apple].
>>> print("Q3:", run_graph_agent("Compare Apple and Tesla revenue in 2024."))
--- 🔍 RETRIEVING ---
🎯 Routing to: both
--- 🧩 PREPARING COMPARE QUERIES ---
   Apple query: Apple revenue in 2024.
   Tesla query: Tesla revenue in 2024.
--- 🍎 RETRIEVING APPLE FOR COMPARISON ---
   Apple query: Apple revenue in 2024.
--- 🍎⚖️ GRADING APPLE FOR COMPARISON ---
   Apple Grade: yes
--- 🚗 RETRIEVING TESLA FOR COMPARISON ---
   Tesla query: Tesla revenue in 2024.
Failed to send telemetry event CollectionQueryEvent: capture() takes 1 positional argument but 3 were given
--- 🚗⚖️ GRADING TESLA FOR COMPARISON ---
   Tesla Grade: yes
--- ⚖️✍️ GENERATING COMPARISON ANSWER ---
Q3: Apple's total net sales for the twelve months ended September 28, 2024, were $391,035 million. Tesla's total rev
enues for the year ended December 31, 2024, were $97,690 million. Apple had significantly higher revenue than Tesla
in 2024 [Source: Apple] [Source: Tesla].
```

```
>>> print("Q4:", run_graph_agent("What was NVIDIA's revenue in 2024?"))
--- 🔍 RETRIEVING ---
🎯 Routing to: none
--- ⚖️ GRADING ---
   Relevance Grade: no (empty context)
--- 🔄 REWRITING QUERY ---
   New Question: What was NVIDIA's total revenue in 2024?
--- 🔍 RETRIEVING ---
🎯 Routing to: none
--- ⚖️ GRADING ---
   Relevance Grade: no (empty context)
--- 🔄 REWRITING QUERY ---
   New Question: What was NVIDIA's total revenue for the fiscal year 2024?
--- 🔍 RETRIEVING ---
🎯 Routing to: none
--- ⚖️ GRADING ---
   Relevance Grade: no (empty context)
   (Max retries reached, generating anyway...)
--- ✍️ GENERATING ---
Q4: I don't know.
```

Chunk size = 4000

```
>>> print("Q1:", run_graph_agent("What was Apple's revenue in 2024?"))
--- 🔍 RETRIEVING ---
🎯 Routing to: apple
   Query for apple: What was Apple's revenue in 2024?
Failed to send telemetry event CollectionQueryEvent: capture() takes 1 positional argument but 3 were given
--- ⚖️ GRADING ---
   Relevance Grade: yes
--- ✍️ GENERATING ---
Q1: Apple's revenue in 2024 was $391,035 million [Source: Apple].
>>> print("Q2:", run_graph_agent("How much did Apple spend on new tech in 2024?"))
--- 🔍 RETRIEVING ---
🎯 Routing to: apple
   Query for apple: How much did Apple spend on new tech in 2024?
--- ⚖️ GRADING ---
   Relevance Grade: no
--- 🔄 REWRITING QUERY ---
   New Question: What were Apple's research and development expenses in 2024?
--- 🔍 RETRIEVING ---
🎯 Routing to: apple
   Query for apple: What were Apple's research and development expenses in 2024?
--- ⚖️ GRADING ---
   Relevance Grade: yes
--- ✍️ GENERATING ---
Q2: Apple's research and development expenses in 2024 were $31,370 million [Source: Apple].
>>> print("Q3:", run_graph_agent("Compare Apple and Tesla revenue in 2024."))
--- 🔍 RETRIEVING ---
🎯 Routing to: both
--- 🧩 PREPARING COMPARE QUERIES ---
   Apple query: Apple revenue in 2024.
   Tesla query: Tesla revenue in 2024.
--- 🍎 RETRIEVING APPLE FOR COMPARISON ---
   Apple query: Apple revenue in 2024.
--- 🍎⚖️ GRADING APPLE FOR COMPARISON ---
   Apple Grade: yes
--- 🚗 RETRIEVING TESLA FOR COMPARISON ---
   Tesla query: Tesla revenue in 2024.
Failed to send telemetry event CollectionQueryEvent: capture() takes 1 positional argument but 3 were given
--- 🚗⚖️ GRADING TESLA FOR COMPARISON ---
   Tesla Grade: yes
--- ⚖️✍️ GENERATING COMPARISON ANSWER ---
Q3: Apple's total net sales for the twelve months ended September 28, 2024, were $391,035 million, while Tesla's tot
al revenues for the year ended December 31, 2024, were $97,690 million. Apple had significantly higher revenue than
Tesla in 2024 [Source: Apple] [Source: Tesla].
```

```
>>> print("Q4:", run_graph_agent("What was NVIDIA's revenue in 2024?"))
--- 🔍 RETRIEVING ---
🎯 Routing to: none
--- ⚖️ GRADING ---
   Relevance Grade: no (empty context)
--- 🔄 REWRITING QUERY ---
   New Question: What was NVIDIA's total revenue in 2024?
--- 🔍 RETRIEVING ---
🎯 Routing to: none
--- ⚖️ GRADING ---
   Relevance Grade: no (empty context)
--- 🔄 REWRITING QUERY ---
   New Question: What was NVIDIA's total revenue for the fiscal year 2024?
--- 🔍 RETRIEVING ---
🎯 Routing to: none
--- ⚖️ GRADING ---
   Relevance Grade: no (empty context)
   (Max retries reached, generating anyway...)
--- ✍️ GENERATING ---
Q4: I don't know.
```

綜合本次實驗結果，1000、2000 與 4000 在整體回答表現上的差異並不算非常明顯，但從理論與文件結構來看，它們各自代表不同的取捨方向。若問題偏向單一明確事實查詢，小 chunk 較有利於精準檢索；若問題涉及大型財務表格，特別是需要保留較完整表格上下文時，2000 或 4000 會更有優勢。

綜合本次實驗結果，1000、2000 與 4000 在整體回答表現上的差異並不算非常明顯，但從理論與文件結構來看，它們各自代表不同的取捨方向。若問題偏向單一明確事實查詢，小 chunk 較有利於精準檢索；若問題涉及大型財務表格，特別是需要保留較完整表格上下文時，2000 或 4000 會更有優勢。

本次 chunk size benchmark 的重點不只是比較 1000、2000 與 4000 哪一個結果較好，更重要的是理解不同 chunk size 對大型表格問答時的上下文保留能力差異，以及 Context Precision 與 Context Completeness 之間的取捨。

LangGraph 與 LangChain 比較

本專案同時實作了 LangChain Legacy Agent 與 LangGraph Agent，兩者在設計理念與實際表現上有明顯差異。

1. LangChain 的特性

   LangChain ReAct agent 的優點是架構較簡單，實作門檻也較低。它可以透過 Thought / Action / Observation 的流程逐步使用 retriever tool，對於單純的問答任務已具備基本效果，因此很適合作為 baseline。

   不過當系統需要更明確地控制流程時，LangChain 的限制就會逐漸浮現。例如本作業要求的 router、grader、rewriter，以及比較題的雙支線檢索流程，若全部都放在單一 agent 內處理，整體控制會變得較不直觀，也較難保證每一步都能穩定執行。

2. LangGraph 的特性

   相較之下，LangGraph 雖然實作上較複雜，但它能夠清楚表示每一個節點與節點之間的條件轉移，因此更適合需要多步驟判斷與自我修正的 RAG 系統。

在本次作業中，LangGraph 的優勢主要體現在以下幾點：
(1)可明確做問題 route，決定該查 Apple、Tesla 或雙方。
(2)可先用 grader 判斷檢索結果是否足夠，再決定是否改寫問題。
(3)可針對 compare 題設計 Apple 與 Tesla 雙支線流程。
(4)可在資訊不足時更穩定地回覆 I don't know，降低 hallucination 風險。

綜合來說，LangChain 比較像是容易上手的 baseline；而 LangGraph 則更適合本作業這種需要 intelligent routing、self-correction 與 multi-document comparison 的任務。因此在最終成果上，我認為 LangGraph 是更完整也更可靠的解法。

從實作角度來看，LangChain 偏向由單一 agent 在 ReAct loop 中動態決定下一步；LangGraph 則是由開發者先定義節點、state 與條件邊，因此在流程控制、錯誤修正與 compare 題處理上更清楚且更可控。

討論

在實作完成後，我觀察到本系統已能穩定處理多種題型。對於 compare revenue 這類比較題，系統可以透過 Apple / Tesla 雙支線流程分別檢索與驗證資料，再整合成最終答案。對於 irrelevant 題，例如詢問 NVIDIA 的財務資訊時，系統也能安全回覆 I don't know，而不會產生與資料集無關的幻覺回答。

從實驗結果來看，embedding model 的差異不大，chunk size 的差異也不算明顯。這表示在目前資料規模較小、主題較集中的情況下，系統整體設計可能比單純更換 embedding model 更影響最終表現。尤其是 router、grader 與 rewriter 這些結構性模組，對系統穩定性與可控性帶來了更直接的幫助。

此外，我認為 compare 題的雙支線設計是本次作業中最關鍵的改進之一。若沒有將 Apple 與 Tesla 分開處理，很容易發生只找到一邊資料卻仍硬做比較的問題。透過雙支線設計，系統能更嚴格確認兩邊證據是否都足夠，再進行最終回答。

結論

本次作業中，我成功完成了 Autonomous Multi-Doc Financial Analyst 的實作，並同時建立了 LangChain 與 LangGraph 兩種版本作為比較。最終的 LangGraph 系統已具備 Intelligent Router、Relevance Grader、Query Rewriter、Final Generator，以及 compare 題雙支線處理能力。

除了完成系統功能外，我也針對兩個 embedding model 與三種 chunk size 進行比較，並分析其對檢索與回答表現的影響。雖然本次實驗中差異並不算大，但仍可看出不同設定在 precision 與 completeness 上的取捨。

整體而言，LangGraph 相較於 LangChain 更適合本作業的需求，因為它能更清楚地控制流程、加入自我修正機制，並在面對比較題與無關題時提供更穩定且可靠的表現。

系統流程圖

一般題:
Question → Router / Retrieve → Grade → Rewrite(if needed) → Retrieve → Generate

比較題:
Question → Router(both) → Prepare Compare Queries
→ Apple branch (Retrieve → Grade → Rewrite if needed → Retrieve)
→ Tesla branch (Retrieve → Grade → Rewrite if needed → Retrieve)
→ Compare Generate